

# Module 5: Data Visualization with Power BI

## Section A: Concept Application — Model Answers

### Question 1: Connection Modes (Import Mode vs. DirectQuery)

- **Recommendation & Conceptual Difference:**

Import Mode loads and compresses a snapshot copy of the data directly into Power BI's in-memory VertiPaq engine, whereas DirectQuery leaves data at the source and sends native SQL/source queries on the fly whenever a visual renders. For historical sales trend analysis across two years, **Import Mode** is recommended because it provides lightning-fast analytical speed, rich time-intelligence capabilities, and full DAX function support. For real-time order tracking, **DirectQuery** (or a Composite Model configuring operational tables in DirectQuery and historical tables in Import Mode) is recommended so that dispatchers can see live status updates every few minutes without hitting scheduled refresh boundaries.

- **Trade-Offs:**

Import Mode delivers optimal query performance and modeling flexibility, but it requires scheduled refreshes (introducing latency) and consumes capacity RAM. DirectQuery avoids data duplication and provides near-real-time visibility, but it places a continuous query load on the underlying transactional database, limits advanced DAX functions, and can cause visual latency during peak dashboard usage.

### Question 2: Power Query Transformation Pipeline

- **Transformation Steps in Sequence:**

1. **Trim & Standardize Text:** Select the City column, navigate to **Transform > Format > Trim** to eliminate leading and trailing spaces, followed by **Format > Capitalize Each Word** to resolve case discrepancies (e.g., standardizing 'ahmedabad' and 'Ahmedabad ' into 'Ahmedabad').
2. **Split Customer & Phone:** Select the combined CustomerName\_Phone column, go to **Home/Transform > Split Column > By Delimiter**, select the hyphen (-) as the separator, and rename the resulting output columns to CustomerName and PhoneNumber.
3. **Remove Blank Rows:** Apply **Home > Remove Rows > Remove Blank Rows** (or filter out nulls/blanks on the primary key column OrderID) to remove empty records.

- **Justification for Power Query over Manual Excel Cleaning:**

Managing transformations inside Power Query Editor records each action as a declarative, repeatable M-code script that executes automatically upon every scheduled or triggered refresh. Manual Excel adjustments are prone to human omission, lack auditability, risk data

corruption, and break automated reporting pipelines in a production environment.

### Question 3: Star Schema Data Modeling

- **Model Structure, Entities & Cardinality:**
  - **Fact Table:** Orders (contains numerical metrics and transactional foreign keys: OrderID, CustomerID, RestaurantID, OrderDate, TotalAmount, DeliveryStatus).
  - **Dimension Tables:** Customers (contextual attributes: CustomerID, Name, City) and Restaurants (contextual attributes: RestaurantID, Name, Cuisine, AvgRating).
  - **Relationships & Cardinality:** Create a **One-to-Many (1 : \*)** relationship from Customers[CustomerID] (1) to Orders[CustomerID] (), *and another One-to-Many (\$1:\$) relationship from Restaurants[RestaurantID] (1) to Orders[RestaurantID] (\*)*. Both relationships must be set to a **Single** cross-filter direction pointing toward the Orders fact table.
- **Efficiency Advantage over Flat Tables:**

A Star Schema separates high-cardinality transactional data from low-cardinality dimensional attributes, allowing the VertiPaq engine to compress dictionary values far more effectively. Furthermore, it creates simple, unambiguous filter propagation paths, eliminating circular relationship dependencies and drastically speeding up DAX aggregation scans compared to traversing wide, denormalized flat tables.

### Question 4: Calculated Columns vs. Measures & Filter Context

- **Conceptual Difference & Recommendation:**

A DAX Calculated Column evaluates row-by-row during data refresh inside a **row context** and stores its results permanently in the model's RAM. A DAX Measure stores no physical data at rest; it computes dynamically at visual rendering time within the active **filter context**. A **Measure** (e.g., Total Revenue = SUMX(Orders, Orders[Quantity] \* Orders[UnitPrice]) or SUM(Orders[TotalAmount])) is the correct approach for the KPI card visual because it calculates an aggregated figure on demand rather than storing redundant column data.
- **Filter Context Evaluation:**

When a user selects a value in the City slicer, that selection filters the Customers dimension table down to the selected city. Because a Measure evaluates within the visual filter context, this filter automatically propagates across the one-to-many relationship into the Orders fact table, restricting the rows evaluated by the measure's aggregation to only orders belonging to customers in that city.

### Question 5: Debugging KPI Slicer Ignorance

- **Two Distinct Causes:**
  1. **DAX Measure Overriding Context:** The DAX expression powering the card contains a filter-clearing function—such as CALCULATE([Total Revenue], ALL(Customers)) or REMOVEFILTERS(Customers[City])—which intentionally removes the slicer filter.
  2. **Visual Interactions Disabled:** The visual interaction setting between the City slicer

and the KPI card has been toggled to **None** instead of **Filter**.

- **Diagnostic & Resolution Steps:**

- **Inspect the DAX Formula:** Check the measure definition in the formula bar to see if ALL, ALLEXCEPT, or REMOVEFILTERS is stripping the city filter; replace them with standard context-sensitive aggregations if so.
- **Verify Visual Interactions:** Select the City slicer, go to the **Format** ribbon, click **Edit Interactions**, and verify that the KPI card has the **Filter** icon highlighted rather than the **None** icon.
- **Inspect Visual-Level Filters:** Select the KPI card and check the **Filters Pane** to confirm no hidden visual-level or page-level filter is overriding the city dimension.

## Question 6: Implementing Row-Level Security (RLS)

- **Implementation in Power BI Desktop:**

1. In Power BI Desktop, open the **Modeling** ribbon and click **Manage Roles**.
2. Click **Create** to define two distinct roles: City\_Ahmedabad and City\_Surat.
3. Under the Customers table (or whichever dimension contains city attributes), write the respective DAX filter expression:
  - For City\_Ahmedabad: [City] = "Ahmedabad"
  - For City\_Surat: [City] = "Surat"
4. Test and validate the rules locally by selecting **View as Roles** in the Modeling ribbon, checking each role, and verifying that visual totals match the respective regional data before publishing.

- **Power BI Service Role Assignment:**

1. Publish the report to the target workspace in Power BI Service.
2. Locate the published semantic model (dataset), click the three dots (...), and select **Security**.
3. Select the City\_Ahmedabad role, enter the corporate email addresses or Entra security groups for Ahmedabad regional managers, and click **Add**. Repeat this procedure for the City\_Surat role and click **Save**.